

AUTOMATION TESTING MASTERY

Dropdown & Alert Handling

A comprehensive guide to automating standard selects, dynamic UI dropdowns, and browser JavaScript alerts cleanly and reliably.

PART I

Dropdown Handling Strategies

Mastering standard HTML select elements and modern custom framework components.

Standard vs. Custom Dropdowns

Standard Dropdowns

Built using native HTML and tags.

Directly supported by browser test automation tool APIs like Selenium's `Select` class without requiring custom click sequences.

Custom UI Dropdowns

Constructed using modern JS frameworks (React, Angular, Vue) with custom or elements.

Cannot use standard select helpers. Requires simulating real user gestures: opening menus, waiting, and selecting items.

Standard Dropdown Methods



selectByVisibleText

Selects an option by its displayed user text.

```
select.selectByVisibleText("India");
```



selectByValue

Selects an option using its HTML value attribute.

```
select.selectByValue("IN");
```



selectByIndex

Selects based on zero-based sequential index position.

```
select.selectByIndex(2);
```


Automating Custom UI Dropdowns

When dealing with non-select elements (such as Bootstrap, React, or Angular dropdowns), standard `Select` wrapper classes throw exceptions.

```
// Step 1: Open dropdown menu driver.findElement(By.id("dropdownMenu")).click(); // Step 2: Locate and select option via XPath driver.findElement
```

Always apply explicit waits to verify the menu expanded state before clicking child menu items.

Best Practices for Dropdowns



Inspect DOM First: Check if the element uses a native tag before picking your automation approach.



Use Explicit Waits: Dynamic dropdowns load items asynchronously. Always wait until options are visible and clickable.



Avoid Index Selection: Prefer visible text or stable unique identifiers over hardcoded indexes which break easily on data updates.



Verify Multi-Select Capability: Always check `isMultiple()` before executing `bulk deselectAll()` operations.

PART II

Alert & Popup Management

Interacting with JavaScript dialog boxes and modal windows seamlessly.

Types of Native JavaScript Alerts



Simple Alert

Displays an informational notification message with only a single **OK** button.



Confirm Box

Presents a decision dialog requesting user confirmation with **OK** and **Cancel** options.



Prompt Box

Requests dynamic textual user input along with standard **OK** and **Cancel** buttons.

Handling Alerts in Automation

Context Switching

Automation tools must switch focus from the main page DOM to the active alert context.

```
Alert alert = driver.switchTo().alert();
```

Interacting with web elements without addressing present alerts throws `UnhandledAlertException`.

Key Alert Methods

```
// Click OK / Confirm alert.accept(); // Click Cancel / Dismiss alert.dismiss();
```


Dynamic Alerts & Synchronization

network requests or user actions. Attempting to switch to a non-existent alert throws a `NoAlertPresentException`.

Alert triggers with explicit wait strategies before execution.

```
driverWait(driver, Duration.ofSeconds(10)); Alert alert = wait.until( ExpectedConditions.alertIsPresent() ); alert.accept();
```


Dropdowns vs. Alerts Comparison

Feature	Standard Dropdowns	JavaScript Alerts
DOM Visibility	Inspectable in DOM tree ()	Outside DOM tree (Native Browser API)
Interaction Mode	Direct API or standard locator clicks	Target context switch (switchTo().alert())
Input Actions	Select options by index, value, or text	Accept, Dismiss, Read Text, Send Text
Common Failures	ElementNotInteractableException	NoAlertPresentException

Questions & Answers

Thank you for exploring Dropdown and Alert Handling!

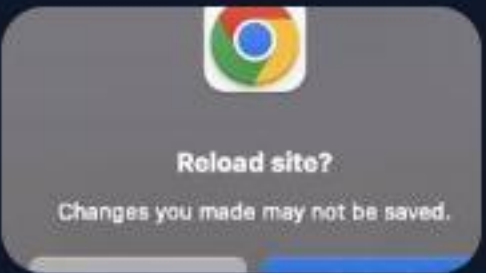
 Ready for discussion & code review

Image Sources



<https://s3.amazonaws.com/blog-assets-production.testsigma.com/blogs/wp-content/uploads/2023/10/02123808/image-3-1.png>

Source: testsigma.com



<https://i.sstatic.net/19CfnQC3.png>

Source: stackoverflow.com